

FAST NUCES

Database Systems

Instructor: Mr. Talha Shahid

Project Proposal

CineMatch: Database Design and Integration for an AI-Powered Movie Recommendation System

Team Members

Tayaba (24K-0934)

Wardah Ahmed (24K-1045)

Note: This project is a combined submission integrating the Database Systems course with the Artificial Intelligence course. Humna Fatime (23K-1016) is part of the CineMatch AI team but does not take the Database Systems course and is therefore not included in this submission.

Submission Date: May 3, 2026

1. Project Overview

CineMatch is an AI-powered movie recommendation system originally developed as part of the Artificial Intelligence course. The system uses content-based filtering, collaborative filtering, SVD, and a hybrid recommendation algorithm to suggest personalized movies to users. For the Database Systems course, we are extending CineMatch by designing and implementing a proper relational database backend that will replace the flat file data storage currently used.

The database will store all movie metadata, user information, ratings, genres, and interaction logs in a normalized relational schema. It will be implemented using SQLite for development with a design that is compatible with PostgreSQL for production deployment. The Python application will interact with the database using the sqlite3 library, and all core queries will be written in standard SQL.

2. Problem Statement

The current CineMatch system loads all data directly from CSV files at startup and stores every model output (cosine similarity matrices, user similarity scores, predicted ratings, and the user-movie matrix) as separate pickle files. This works for a one-time demo but breaks down as soon as the system needs to do anything dynamic. Right now, there is no way to record which recommendations a user actually received, no way to track new ratings as users interact with the system, and no way to query relationships between users, movies, and genres without loading everything into memory first.

The collaborative filtering logic recomputes similarity scores over the entire user-movie matrix on every request because there is no persistent layer to store or retrieve intermediate results. Adding a new user or movie requires retraining the model from scratch rather than inserting a row. A relational database solves these problems directly: it provides persistent storage for user interactions and recommendation logs, enables efficient SQL queries over movie and rating data without loading full matrices into memory, and establishes the structured relationships between users, movies, genres, and ratings that the hybrid algorithm depends on but currently has no formal way to enforce.

3. Objectives

- Design a fully normalized relational database schema for the CineMatch system
- Create an ER diagram showing all entities, attributes, and relationships
- Implement the database in SQLite using Python
- Populate the database with the MovieLens 100K dataset (100,000 ratings, 943 users, 1,682 movies)
- Write meaningful SQL queries covering joins, aggregations, subqueries, and filtering
- Integrate the database with the existing CineMatch Python application
- Log all user interactions and recommendation requests to the database
- Demonstrate the system working end to end with live data from the database

4. Proposed Database Schema

The database will consist of six tables organized in a normalized relational structure. The schema is designed to be in Third Normal Form (3NF) to eliminate redundancy and ensure data integrity. A full ER diagram showing all entities, attributes, primary keys, foreign keys, and cardinalities is included as Appendix A at the end of this proposal and will also be submitted as a separate file with the final project.

4.1 Tables Overview

Table	Primary Key	Key Columns	Purpose
users	user_id	age, gender, occupation, zip_code	Stores user profiles from MovieLens
movies	movie_id	title, release_year, imdb_url	Stores movie information
genres	genre_id	genre_name	Lookup table for all genres
movie_genres	movie_id + genre_id	movie_id (FK), genre_id (FK)	Many-to-many: movies to genres
ratings	rating_id	user_id (FK), movie_id (FK), score, rated_at	All user ratings (scale 1 to 5)
interaction_logs	log_id	user_id (FK), query_movie, recommended_movies, created_at	Logs every recommendation request

4.2 Relationships

- One user can have many ratings (one-to-many between users and ratings). This is the core relationship for collaborative filtering: the ratings table is the bridge between users and movies, and all similarity computations are derived from it.
- One movie can have many ratings (one-to-many between movies and ratings). A movie with fewer than a threshold number of ratings is treated as a cold-start item; this count is directly queryable from the ratings table using GROUP BY and HAVING.

- One movie can belong to many genres and one genre can belong to many movies (many-to-many, resolved via the movie_genres junction table). Separating genres into their own lookup table eliminates the repeating columns that appear in the raw MovieLens CSV and brings the schema into 1NF.
- One user can have many interaction logs (one-to-many between users and interaction_logs). The interaction_logs table is the only persistent record of what the system actually recommended, which does not exist anywhere in the current flat-file implementation.
- All foreign key constraints will be enforced with ON DELETE CASCADE. This choice ensures referential integrity automatically: if a user is removed, their ratings and interaction logs are removed with them, preventing orphaned rows. The alternative (RESTRICT) was considered but rejected because it would require manual cleanup in the data population scripts.

4.3 Indexing Strategy

The ratings table will contain 100,000 rows after the MovieLens dataset is loaded, making it the performance-critical table in the schema. Without indexes, any query that filters or joins on user_id or movie_id requires a full table scan. Three indexes will be created to avoid this:

- idx_ratings_user_id on ratings(user_id) — supports user rating history lookups and collaborative filtering joins
- idx_ratings_movie_id on ratings(movie_id) — supports per-movie average rating queries and the top-10 highest rated movies query
- idx_movie_genres_movie_id on movie_genres(movie_id) — supports genre-based filtering joins between movies and genres

5. SQL Queries Planned

The following SQL queries will be implemented and demonstrated as part of the project:

Query	Description	Concepts Used
Top 10 highest rated movies	Average rating per movie, minimum 50 ratings	GROUP BY, HAVING, ORDER BY, AVG
User rating history	All movies a specific user has rated with titles	JOIN, WHERE, ORDER BY
Genre-based movie search	All movies belonging to a specific genre	JOIN, WHERE, LIKE
Most active users	Users who have rated the most movies	GROUP BY, COUNT, ORDER BY
Movies rated by similar users	Find movies rated highly by users with similar tastes	Subquery, JOIN, GROUP BY
Recommendation log analysis	Most frequently recommended movies across all users	GROUP BY, COUNT, ORDER BY
Cold start handling	Find popular movies for new users with no ratings	Subquery, AVG, COUNT, HAVING

Genre popularity	Count of movies per genre sorted by popularity	JOIN, GROUP BY, COUNT
------------------	--	-----------------------

6. Technology Stack

Component	Technology	Purpose
Database	SQLite 3	Lightweight relational database, no server required
Production DB	PostgreSQL 14+	Production-ready deployment option
Language	Python 3.9+	Database interaction and data population scripts
DB Library	sqlite3 (built-in)	Python interface for SQLite
Data Source	MovieLens 100K	Source dataset for populating the database
ORM (optional)	None (raw SQL)	All queries written in standard SQL for learning purposes
Integration	FastAPI + SQLite	Database connected to the CineMatch REST API. Each API endpoint (get recommendations, log interaction, fetch user history) will open a connection using sqlite3, execute the relevant SQL query, and return the result as JSON. The database layer sits between the FastAPI routes and the recommendation logic, replacing the current in-memory matrix lookups with live SQL queries.

7. Project Timeline

Task	Target Date	Status
Database schema design and ER diagram	Apr 23, 2026	In Progress
SQLite database implementation	Apr 24, 2026	Upcoming
Data population from MovieLens dataset	Apr 24, 2026	Upcoming
SQL queries implementation and testing	Apr 25 and 26, 2026	Upcoming
Integration with CineMatch FastAPI backend	Apr 27 and 28, 2026	Upcoming
Project report writing	Apr 29 and 30, 2026	Upcoming
Final submission and presentation	May 3, 2026	Upcoming

8. Team & Responsibilities

Member	ID	DB Responsibilities
Tayaba	24K-0934	Schema design, ER diagram, data population scripts
Wardah Ahmed	24K-1045	SQL queries, FastAPI database integration, project report